

AdventureWorks Sales Performance Dashboard

Project Objective:

The objective of this project is to perform an end-to-end analysis of the AdventureWorks sales dataset to uncover meaningful business insights and visualize sales performance. The project aims to identify sales trends, analyze customer behavior, evaluate product performance, and understand territory-wise order distribution through the use of data analysis and business intelligence tools.

Problem Statement:

Businesses generate large volumes of sales data, making it difficult to extract meaningful insights without proper analysis. This project focuses on transforming raw sales data into actionable information to support data-driven decision-making and improve business performance.

Tools and Technologies Used:

1. Microsoft Excel
 - a. Data Cleaning
 - b. Data Integration
 - c. Pivot Table Analysis
2. Python
 - a. Exploratory Data Analysis (EDA)
 - b. Data Visualization
3. Libraries Used
 - a. Pandas
 - b. Matplotlib
4. SQLite
 - a. Data Querying
 - b. Aggregation Analysis
5. Power BI

- a. Interactive Dashboard Development
- b. DAX Calculated Columns and Measures
- c. Data Visualization

Dataset Description:

The project utilizes the AdventureWorks sales dataset, which contains information related to customer orders, products, territories, order dates, and order quantities. Three separate datasets were combined into a single consolidated dataset named SalesData to facilitate analysis.

Project Workflow

The project was carried out in the following phases:

1. Data Cleaning and Integration Using Microsoft Excel.
2. Data Analysis Using Excel Pivot Tables.
3. Python-Based Exploratory Data Analysis (EDA).
4. SQL-Based Analysis Using SQLite.
5. Interactive Dashboard Development Using Power BI.
6. Business Insights and Conclusions.

Expected Outcome

The project aims to provide a comprehensive view of sales performance by identifying yearly and monthly trends, top-performing products, valuable customers, and territory-wise order distribution. The insights obtained from this analysis can support strategic business decisions and improve overall operational efficiency.

Excel

Data Cleaning and Data Integration Using Microsoft

The project began with the preprocessing and preparation of the raw sales data using Microsoft Excel. Three separate data files were combined into a single consolidated dataset named **SalesData** to facilitate efficient analysis.

Data Integration

1. Combined the three sales-related datasets into a single unified dataset.
2. Ensured consistency among columns and records during the merging process.
3. Created a consolidated dataset to simplify further analysis and visualization.

Data Cleaning Activities

The following five-step data cleaning process was performed to improve data quality and ensure consistency:

1. Duplicate Value Check

- a) Inspected the dataset for duplicate records using Excel's Remove Duplicates feature.
- b) Confirmed that no duplicate values were present in the dataset.

2. Missing Value Check

- a) Examined the dataset for blank or missing values.
- b) Verified that all records contained complete information.

3. Invalid Value Check

- a) Checked numerical columns for invalid or negative values.
- b) Confirmed that no abnormal values were present.

4. Data Type and Format Validation

- a) Reviewed the data types of all columns.
- b) Converted identifier columns such as ProductKey into appropriate numeric formats.
- c) Ensured consistency in date and numerical formats throughout the dataset.

5. Data Integrity Verification

Performed final validation to ensure the dataset was clean, accurate, and suitable for analysis.

This preprocessing phase established a reliable and consistent dataset, providing a strong foundation for subsequent analysis using Excel Pivot Tables, Python, SQL, and Power BI.

Data Analysis Using Excel Pivot Tables

1. Year-wise order analysis to understand annual sales performance.
2. Month-wise order analysis to identify seasonal trends and demand patterns.
3. Territory-wise analysis to determine high-performing regions.
4. Top customer analysis to identify customers with the highest number of orders.
5. Top product analysis to identify the most frequently ordered products.

These analyses provided an initial understanding of the sales data and helped uncover important trends that guided further exploration using Python, SQL, and Power BI.

References:

Row Labels	Count of OrderNumber
9	12409
4	11463
1	8267
6	6875
10	6423
8	5289
7	5239
5	34
2	27
3	20
Grand Total	56046

Row Labels	Count of OrderNumber
11300	64
11185	61
11262	59
11277	57
11091	56
11330	56

Row Labels	Count of OrderNumber
477	3983
480	2952
528	2846
529	2173
214	2099
223	2062
220	1995
485	1975
215	1940
478	1896
479	1668
530	1364
537	1305
536	1059
538	957
539	868
377	867
511	863

Python-Based Exploratory Data Analysis (EDA)

Python was utilized to perform Exploratory Data Analysis (EDA) on the sales dataset. The analysis was carried out using Pandas for data manipulation and Matplotlib for data visualization. The objective of this phase was to understand sales patterns, identify top-performing entities, and extract meaningful insights from the data.

Libraries Used

1. Pandas
2. Matplotlib

Data Preparation

1. Loaded the Excel dataset into a Pandas DataFrame.
2. Inspected the dataset structure and data types.
3. Removed unnecessary columns.

4. Converted the OrderDate column into datetime format.
5. Created additional columns such as Year and Month for time-based analysis.

Exploratory Data Analysis(EDA)

I have Done

1. Yearly order trend analysis.
2. Monthly order trend analysis.
3. Top 10 products based on order count.
4. Top customers based on order count.
5. Territory-wise order distribution.
6. Product quantity analysis.

The Python EDA phase helped uncover patterns and trends in the dataset and provided a strong foundation for further analysis using SQL and Power BI.

References:

```
[1]: import pandas as pd
import matplotlib.pyplot as plt

[2]: pip install openpyxl

Requirement already satisfied: openpyxl in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from openpyxl) (2.0.0)
Note: you may need to restart the kernel to use updated packages.

[3]: df = pd.read_excel("D:\pr3\Dataset\Sales Data.xlsx")
```

```
[5]: df.describe()
```

	OrderDate	StockDate	ProductKey	CustomerKey	TerritoryKey	OrderLineItem	OrderQuantity	Column1	_1
count	56046	56046	56046.000000	56046.000000	56046.000000	56046.000000	56046.000000	0.0	1.0
mean	2021-12-16 19:51:03.526389	2021-10-10 11:31:25.750990	439.357082	18843.645559	6.254666	1.903151	1.501873	NaN	1.0
min	2020-01-01 00:00:00	2019-09-11 00:00:00	214.000000	11000.000000	1.000000	1.000000	1.000000	NaN	1.0
25%	2021-10-06 00:00:00	2021-07-30 00:00:00	362.000000	14016.000000	4.000000	1.000000	1.000000	NaN	1.0
50%	2022-01-11 00:00:00	2021-11-05 00:00:00	479.000000	18157.000000	7.000000	2.000000	1.000000	NaN	1.0
75%	2022-04-12 00:00:00	2022-02-03 00:00:00	529.000000	23425.750000	9.000000	2.000000	2.000000	NaN	1.0
max	2022-06-30 00:00:00	2022-06-15 00:00:00	606.000000	29483.000000	10.000000	8.000000	3.000000	NaN	1.0
std	NaN	NaN	118.305813	5412.449843	2.958005	1.021006	0.612379	NaN	NaN

```
5]: df.columns
df=df.drop(columns=['Column1' , '_1'],)
```

```
7]: df.info()
<class 'pandas.DataFrame'>
RangeIndex: 56046 entries, 0 to 56045
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Source.Name      56046 non-null  str
1   OrderDate        56046 non-null  datetime64[us]
2   StockDate        56046 non-null  datetime64[us]
3   OrderNumber      56046 non-null  str
4   ProductKey       56046 non-null  int64
5   CustomerKey      56046 non-null  int64
6   TerritoryKey     56046 non-null  int64
7   OrderLineItem    56046 non-null  int64
8   OrderQuantity    56046 non-null  int64
dtypes: datetime64[us](2), int64(5), str(2)
memory usage: 3.8 MB
```

Name: count, dtype: int64

```
##Order By Month
df['Month']=df['OrderDate'].dt.month_name()
df['Month'].value_counts()
```

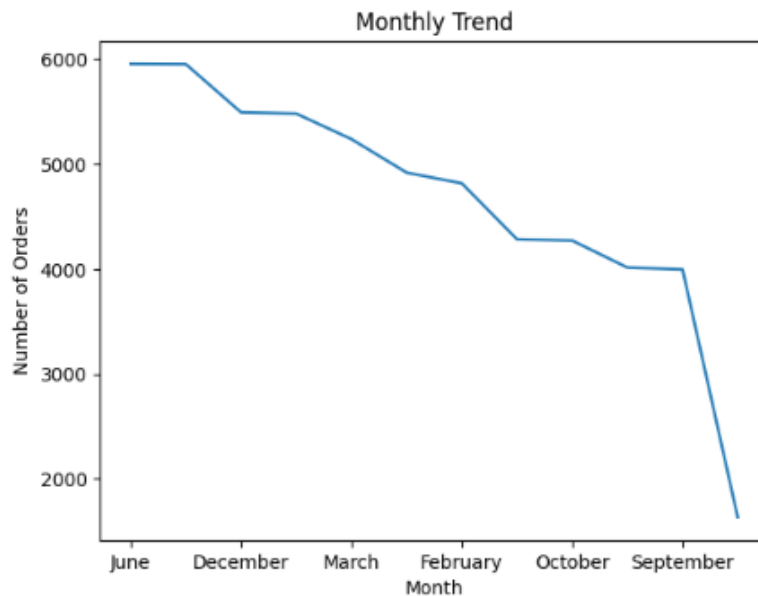
```
: Month
June      5954
May       5951
December  5492
April     5480
March     5237
January   4918
February  4816
November  4283
October   4271
August    4014
September 3996
July      1634
Name: count, dtype: int64
```

```
[14]: month_counts = df['Month'].value_counts()

month_counts.plot(kind='line')

plt.title('Monthly Trend')
plt.xlabel('Month')
plt.ylabel('Number of Orders')

plt.show()
```



```
## Business Insights
## Orders by year
year_orders = df.groupby('Year')['OrderNumber'].count()
print(year_orders)

Year
2020    2630
2021    23935
2022    29481
Name: OrderNumber, dtype: int64
```

```
## Top Products
product_orders = df.groupby('ProductKey')['OrderNumber'].count()
print(product_orders.sort_values(ascending=False).head(10))

ProductKey
477    3983
480    2952
528    2846
529    2173
214    2099
223    2062
220    1995
485    1975
215    1940
478    1896
Name: OrderNumber, dtype: int64
```

```
## Top Customers
customer_orders = df.groupby('CustomerKey')['OrderNumber'].count()
print(customer_orders.sort_values(ascending=False).head(10))

CustomerKey
11300    64
11185    61
11262    59
11277    57
11330    56
11331    56
11091    56
11566    55
11287    54
11223    54
Name: OrderNumber, dtype: int64
```

SQL ANALYSIS

Total Orders:

```
SELECT COUNT(LineNumber) AS Total_Orders
```

```
FROM SalesData;
```

	Total_Orders
1	56046

Orders by Year:

```
SELECT substr(OrderDate,-4) AS Year,
```

```
       COUNT(LineNumber) AS Total_Orders
```

```
FROM SalesData
```

```
GROUP BY Year
```

```
ORDER BY Year;
```

	Year	Total_Orders
1	2020	2630
2	2021	23935
3	2022	29481

Order by Territory:

```
SELECT TerritoryKey, COUNT(LineNumber) AS Total_Orders
```

```
FROM SalesData
```

```
GROUP BY TerritoryKey
```

```
ORDER BY Total_Orders DESC;
```

	TerritoryKey	Total_Orders
1	9	12409
2	4	11463
3	1	8267
4	6	6875
5	10	6423
6	8	5289
7	7	5239
8	5	34

Top 10 Products:

```
SELECT ProductKey,  
        COUNT(OrderNumber) AS Total_Orders  
FROM SalesData  
GROUP BY ProductKey  
ORDER BY Total_Orders DESC  
LIMIT 10;
```

	ProductKey	Total_Orders
1	477	3983
2	480	2952
3	528	2846
4	529	2173
5	214	2099
6	223	2062
7	220	1995
8	485	1975
9	215	1940
10	478	1896

Top 10 Customers:

```
SELECT CustomerKey,  
        COUNT(OrderNumber) AS Total_Orders  
FROM SalesData  
GROUP BY CustomerKey  
ORDER BY Total_Orders DESC  
LIMIT 10;
```

	CustomerKey	Total_Orders
1	11300	64
2	11185	61
3	11262	59
4	11277	57
5	11331	56
6	11330	56
7	11091	56
8	11566	55
9	11287	54
10	11223	54

Total Quantity Sold by Product:

```
SELECT ProductKey,
       SUM(OrderQuantity) AS Total_Quantity
FROM SalesData
GROUP BY ProductKey
ORDER BY Total_Quantity DESC
LIMIT 10;
```

	ProductKey	Total_Quantity
1	477	7967
2	480	5898
3	528	5678
4	529	4327
5	223	4151
6	485	3960
7	478	3810
8	479	3329
9	530	2740
10	536	2119

Average Order Quantity:

```
SELECT AVG(OrderQuantity) AS Average_Quantity
FROM SalesData;
```

	Average_Quantity
1	1.50187346108554

DashBoard Creation

Power BI Dashboard Development

Power BI was used to create an interactive dashboard for analyzing sales performance. After importing the cleaned dataset, several calculated columns and measures were created using DAX to support time-based analysis and key performance indicators.

Calculated Columns Created

1. **Year** = YEAR('SalesData'[OrderDate])
2. **Month** = FORMAT('SalesData'[OrderDate], "MMMM")
3. **Month_Number** = MONTH('SalesData'[OrderDate])

The Month_Number column was used to sort the Month column chronologically.

Measures Created

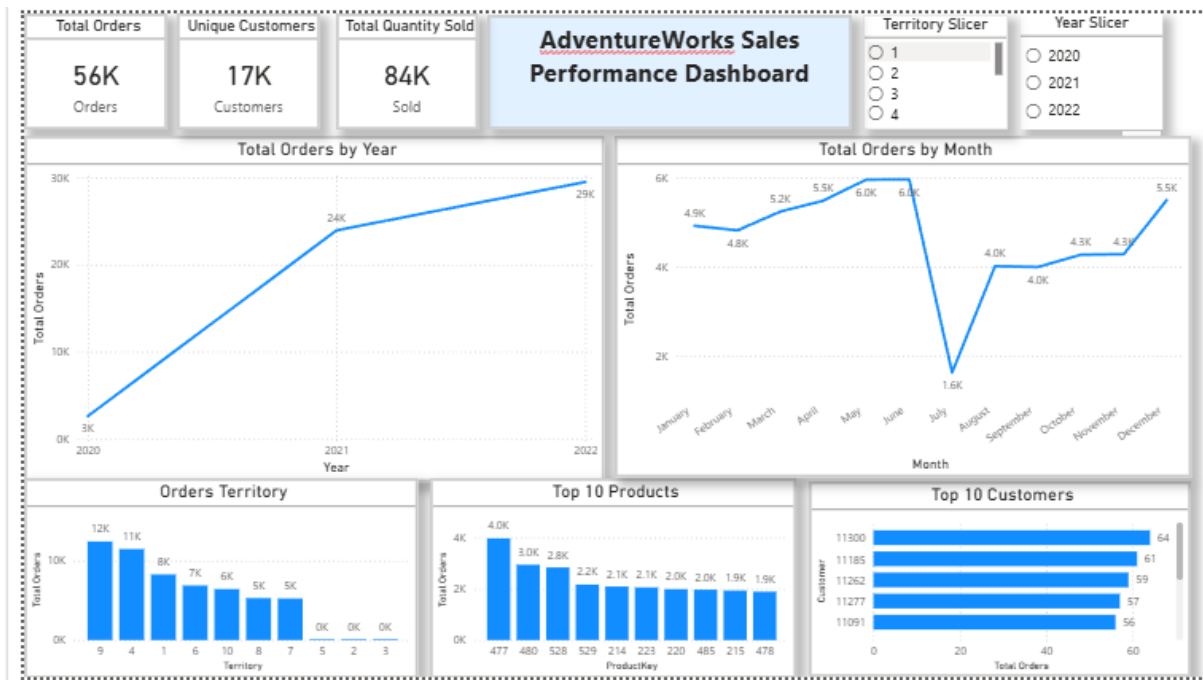
1. **Total Orders** = COUNT('SalesData'[OrderNumber])
2. **Total Quantity Sold** = SUM('SalesData'[OrderQuantity])
3. **Unique Customers** = DISTINCTCOUNT('SalesData'[CustomerKey])

Dashboard Visualizations

The following visualizations were created to analyze sales performance:

1. KPI Cards for Total Orders, Total Quantity Sold, Unique Customers, and Unique Products.
2. Line Chart to visualize yearly order trends.
3. Column Chart to analyze monthly order patterns.
4. Top 10 Products by order volume.
5. Top 10 Customers based on number of orders.
6. Territory-wise order distribution.
7. Interactive slicers for Year and Territory to enable dynamic filtering.

The dashboard provides a comprehensive view of sales trends, customer behavior, product performance, and territory performance, allowing users to derive meaningful business insights from the data.



Business Insights

Based on the analysis performed using Excel, Python, SQL, and Power BI, the following key business insights were derived:

1. Strong Year-over-Year Growth

The sales data revealed a consistent increase in the number of orders over the years, with 2022 recording the highest order volume. This indicates sustained business growth and increasing customer demand.

2. Presence of Seasonal Demand Patterns

Monthly order analysis showed noticeable variations in sales activity throughout the year. Certain months experienced significantly higher order volumes, indicating the existence of seasonal demand patterns that can assist in inventory and marketing planning.

3. High-Performing Territories Contribute Significantly to Sales

Territory-wise analysis highlighted that a few territories accounted for a substantial portion of total orders. These regions represent key markets and provide opportunities for targeted business strategies and expansion.

4. A Small Number of Products Drive Most Orders

Product analysis showed that only a limited number of products consistently generated the highest number of orders. Focusing on these high-performing products can improve inventory management and maximize business performance.

5. Customer Retention Plays a Vital Role

Customer analysis revealed that several customers repeatedly placed orders, demonstrating the importance of repeat customers. Strengthening customer relationships and implementing retention strategies can contribute to long-term business growth.

Conclusion

This project successfully transformed raw sales data into meaningful business insights through the application of Microsoft Excel, Python, SQL, and Power BI. The interactive dashboard and analytical findings provide valuable information regarding sales trends, customer behavior, product performance, and territory performance. These insights can support data-driven decision-making and help improve overall business efficiency.